

Technische handleiding & data-recovery

Voor technisch beheer · Intern Studentbeheersysteem (SIS) · IUASR

Dit document is bestemd voor **technisch personeel/beheerders**. Het beschrijft de architectuur, het maken van back-ups en de **herstelprocedure**. Bewaar het vertrouwelijk.

1. Architectuur & stack

- **Applicatie:** PHP + Laravel (server-rendered; geen kale PHP/WordPress).
- **Database:** MySQL/MariaDB (InnoDB, echte foreign keys, surrogaatsleutels).
- **Authenticatie:** Microsoft Entra ID (SSO/OIDC) — nooit een eigen login bouwen. In de ontwikkelomgeving een tijdelijke dev-login.
- **Netwerk:** draait intern (intranet), IP-beperkt, gescheiden van het publieke aanmeldportaal.
- **Gevoelige data:** BSN en rekeningnummer versleuteld (Laravel-encryptie met **APP_KEY**); inzage/mutatie gelogd (audit-log).

2. Omgeving (referentie)

PHP	~/php/8.3/php.exe
Composer	~/bin/composer.phar
Database-server	MariaDB (portable) — ~/mariadb/mariadb-11.4.9-win64/bin/
DB-host / poort	127.0.0.1 : 3307
Database / gebruiker	iuasr_sis / iuasr_sis
Configuratie	.env (in de projectmap) — bevat DB-gegevens en APP_KEY

APP_KEY is kritiek. Zonder de originele APP_KEY zijn de versleutelde velden (BSN, rekeningnummer) onherstelbaar. De sleutel zit in .env en wordt meegenomen in de back-up.

E-mail (resultaten mailen)

De examencommissie kan definitieve resultaten per e-mail naar studenten sturen. In

ontwikkeling staat `MAIL_MAILER=log`: e-mails worden naar `storage/logs/laravel.log` geschreven, er gaan GEEN echte e-mails uit (AVG, synthetische data). In **productie** configureert u de IUASR-mailserver in `.env`:

```
MAIL_MAILER=smtp
MAIL_HOST=<smtp.iuasr.nl>
MAIL_PORT=587
MAIL_USERNAME=<gebruiker>
MAIL_PASSWORD=<wachtwoord>
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=noreply@iuasr.nl
MAIL_FROM_NAME="IUASR Studentenzaken"
```

Elke student ontvangt individueel de eigen (ondertekende) cijferlijst als bijlage; verzending wordt gelogd. Overweeg voor grote aantallen een queue (`QUEUE_CONNECTION + worker`).

3. Back-up maken

Een volledige back-up wordt gemaakt via de webapplicatie:

1. Log in als **Beheerder**.
2. Ga naar **Beheer → Back-up & herstel**.
3. Geef een sterk **wachtwoord** op (minimaal 8 tekens) en bevestig.
4. Klik op **Back-up genereren & downloaden**. U ontvangt een met AES-256 versleutelde ZIP.

De ZIP bevat: `database.sql` (volledige dump), de applicatiebroncode en webpagina's, `.env` (incl. `APP_KEY`) en de geüploade bestanden (`storage/app`). Niet inbegrepen: `vendor/`, `.git/` en de referentiemap `IUASR/`. Het wachtwoord wordt **nergens opgeslagen**; downloaden wordt ge-audit-logd.

Advies: bewaar back-ups versleuteld op een beveiligde, interne locatie en hanteer een bewaarschema (bijv. wekelijks + vóór elke update). Overweeg een periodieke, geautomatiseerde back-up naar een netwerkschijf.

4. Herstelprocedure (recovery)

Terugzetten is een bewuste beheerhandeling en gebeurt **niet** vanuit de draaiende applicatie (die kan zichzelf niet veilig overschrijven en een database-restore wist bestaande data). Voer de

stappen uit op de server.

Stap 1 — Archief uitpakken

De Windows Verkenner kan een AES-versleutelde ZIP **niet** openen. Gebruik het meegeleverde commando (of 7-Zip/WinRAR):

```
~/php/8.3/php.exe artisan backup:uitpakken "pad/naar/iuasr-sis-backup-JJJJMMDD-UUMM.zip" --doel="pad/naar/hersteld"
```

U wordt om het wachtwoord gevraagd (of geef --wachtwoord=...). Het commando verifieert het wachtwoord en pakt uit naar de doelmap.

Stap 2 — Bestanden plaatsen

Plaats de uitgepakte bestanden in de webroot/projectmap van de (interne) server. Zet ook storage/app (geüploade documenten) terug.

Stap 3 — Afhankelijkheden herstellen

```
~/bin/composer.phar install --no-dev --optimize-autoloader
```

Stap 4 — Database terugzetten

Maak (indien nodig) een lege database en importeer de dump. De dump bevat zowel het schema als de data (DROP/CREATE/INSERT):

```
~/mariadb/mariadb-11.4.9-win64/bin/mariadb.exe -h 127.0.0.1 -P 3307 -u iuasr_sis -p iuasr_sis < database.sql
```

Let op: dit **overschrijft** de bestaande gegevens in de database iuasr_sis. Maak eerst een verse back-up van de huidige stand als die nog waarde heeft.

Stap 5 — Configuratie controleren

- Controleer .env: DB_*-gegevens en APP_URL.
- **Laat APP_KEY ongewijzigd** (gelijk aan de back-up), anders zijn BSN/rekeningnummer niet te ontsleutelen.

Stap 6 — Cache legen & controleren

```
~/php/8.3/php.exe artisan optimize:clear
```

Een php artisan migrate is **niet** nodig: de dump bevat het volledige schema. Controleer tot slot of de applicatie start en of inloggen werkt.

5. Losse database-restore (zonder volledige recovery)

Alleen de gegevens terugzetten (code ongewijzigd)? Pak het archief uit (stap 1) en voer alleen stap 4 uit met `database.sql`.

6. AVG & beveiliging

- Back-ups bevatten alle persoonsgegevens én de encryptiesleutel — uitsluitend versleuteld en intern bewaren; niet e-mailen of naar buiten brengen.
- Echte productiedata alleen in de laatste fase, onder toezicht van de Functionaris Gegevensbescherming.
- Inzage/mutatie van cijfers en BSN wordt gelogd (audit-log, alleen-lezen voor Beheer).
- Rolscheiding wordt server-side afgedwongen; wijzig dit niet zonder reden.
- **Directie is opleidinggebonden.** De koppeltabel `directie_opleidingen` (user ↔ opleiding) bepaalt welke studenten, cijfers en rapporten een directielid ziet. Beheer wijst dit toe via **Gebruikers & rollen → Directie — opleidingtoewijzing**. Zonder toewijzing ziet een directielid **niets** (need-to-know). Een dubbel ingeschreven student is zichtbaar voor de directie van elke opleiding waarin hij/zij actief is. De filtering loopt via `User::opleidingIds()`, `Student::scopeZichtbaarVoor()` en per-opleiding gefilterde statistieken.
- **Presentiegegevens zijn onderwijsinhoudelijk.** Studentenzaken, Financiële Administratie en Beheer hebben géén toegang tot presentielijsten of aanwezigheidspercentages (Gate presentie-inzien). Registreren mag alleen de docent van het eigen vak (Gate presentie-registreren). Inzage en mutatie worden gelogd.

7. Presentie (aanwezigheidsregistratie)

De docent registreert per college de aanwezigheid; dit is verplicht. Het model is bewust **genormaliseerd**: één regel per student × vak × onderwijsweek — nooit vaste weekkolommen op de inschrijving.

Tabel	<code>presenties</code> (<code>inschrijving_id</code> , <code>vak_id</code> , <code>week</code> , <code>aanwezig</code> , <code>geregistreerd_door_id</code>) met unieke sleutel op (<code>inschrijving_id</code> , <code>vak_id</code> , <code>week</code>).
Regeling	Kolom <code>inschrijvingen.aanwezigheidsregeling_50</code> (boolean). Bewust op de inschrijving , niet op de student: zij geldt per opleiding en per studiejaar en moet bij herinschrijving opnieuw worden toegekend.
Normen	<code>config/sis.php</code> → <code>presentie.weken_per_blok</code> (8), <code>presentie.norm</code> (0.80) en <code>presentie.norm_regeling</code> (0.50).

Ontbrekende registratie is geen afwezigheid. Het percentage wordt berekend over de **geregistreerde** weken. Een week zonder regel telt niet mee — anders zou nalatigheid van de docent op de student worden afgewenteld. Een week geldt pas als “geregistreerd” wanneer alle presentieplichtige deelnemers een waarde hebben.

Vrijgestelde studenten (`vaktoewijzingen.vrijgesteld`) volgen het vak niet en worden bij het opslaan overgeslagen, óók als het formulier voor hen een waarde meestuurt. Wijzigt u `weken_per_blok`, dan blijven bestaande registraties met een hoger weeknummer in de database staan maar verdwijnen zij uit het scherm; ruim ze in dat geval expliciet op.

8. Collegegeld: termijnen

Er is bewust **geen facturentabel**. Het termijnschema wordt volledig afgeleid uit het jaartarief, de betaalregeling en de inschrijvingsduur, zodat het nooit kan verouderen ten opzichte van de inschrijving (bijvoorbeeld na een gewijzigde uitschrijfdatum).

Schema	App\Support\Collegegeldtermijnen — vervalmaanden 9, 11, 1, 3, 5; bedrag = jaarbedrag ÷ n, restje op de laatste termijn.
Regeling	<code>inschrijvingen.betaalregeling</code> : termijnen (5 facturen) of volledig (1 factuur, vervalt 1 september).
Betaling → termijn	<code>betalingen.termijn</code> (1..5, nullable). Leeg = automatisch toerekenen aan de oudste openstaande termijn (FIFO).
Achterstand	Som van het openstaande deel van de termijnen waarvan de vervaldatum verstreken is. Dit stuurt de blokkades op herinschrijven en verklaringen.

Per opleiding (beleid 2026-07-10). Collegegeld hangt aan de INSCHRIJVING, niet aan het studiejaar. Elke inschrijving heeft een eigen termijnschema en eigen betalingen; `Collegegeldstatus::voor()` telt alle inschrijvingen op. Korting staat op `inschrijvingen.korting_percentage` (+ verplichte `korting_reden`); `Collegegeldstatus::jaarbedrag()` geeft het tarief ná korting en is de basis voor het schema. Bij 100% korting levert `Collegegeldtermijnen::voor()` een lege collectie. Betalingen worden nooit tussen opleidingen verrekend. Een achterstand bij één inschrijving zet achterstand op true en blokkeert herinschrijven en verklaringen.

Betalingen zijn muteerbaar. `BetalingController::bijwerken()` en `::verwijderen()` (rollen financien + beheerder) schrijven de oude en nieuwe waarden naar de audit-log (veld: `betaling`). Verwijder deze logging niet: zij is het enige bewijsspoor van mutaties op geldbedragen.

Let op de betekenis van de velden in `Collegegeldstatus::voor()`: `verschuldigd` is het totaal van de niet-vervallen termijnen (bij een lopende inschrijving dus het volle jaarbedrag), `openstaand` is `verschuldigd – betaald` (inclusief termijnen die nog moeten vervallen), en `achterstallig` is het direct opeisbare bedrag. Alleen `achterstallig` bepaalt achterstand.

De kolom `inschrijvingen.betaalwijze` is **vervallen** (zij mengde regeling en betaalwijze) en blijft alleen voor de historie bestaan. De betaalwijze hoort bij een betaling, niet bij de inschrijving.

Bij een beëindigde inschrijving wordt het totaal herrekend naar het pro rata bedrag; termijnen met een vervaldatum ná het einde worden vervallen en de laatste geldende termijn vangt het verschil op. De CSV-import herkent kolommen op **naam** uit de kopregel, met terugval op de klassieke volgorde (`studentnummer;bedrag;datum;betaalwijze;opmerking`) voor oudere bestanden.

9. Curriculum (vakken)

Bron	<code>database/data/curriculum.csv</code> (uit 'vakkenlijst update.xlsx', 2026-07-10), geladen door <code>CurriculumSeeder</code> . Referentiedata, geen persoonsgegevens: hoort in Git.
Herladen	<code>php artisan db:seed --class=CurriculumSeeder</code> — idempotent: matcht op (opleiding, code) en werkt bij. Vakken die niet in de CSV staan blijven ongemoeid.
EC	<code>vakken.ec</code> is <code>decimal(4,1)</code> : halve studiepunten (2,5) komen voor. Nooit naar <code>int</code> casten. Gebruik <code>App\Support\Ec::toon()</code> voor weergave (Nederlandse komma).
Vakcode	Uniek per opleiding: <code>unique(opleiding_id, code)</code> . Elf codes bestaan in zowel ISLTH als PMGV.
Blok	<code>null</code> = het vak loopt het hele studiejaar (stage, scriptie). Views die over blok 1..4 itereren moeten blok <code>null</code> apart tonen.
Keuzevak	<code>vakken.keuzevak</code> . <code>Vaktoewijzer</code> slaat keuzevakken over; <code>Overgangsbeoordeling</code> telt alleen de keuzevakken mee die aan de inschrijving zijn toegewezen.

Synthetische vakken horen niet naast het echte curriculum. De voorbeeldvakken met code `ISLTH-*` zijn verplaatst naar `SynthetischVakSeeder`, die alleen door de testsuite wordt gebruikt en

bewust NIET in DatabaseSeeder staat. Stonden zij actief naast het echte curriculum, dan telden zij mee in de EC-totalen per leerjaar (jaar 1 werd 94 i.p.v. 60 EC) en werden zij automatisch aan elke ISLTH-student toegewezen.

Nog te doen: de nieuwe vakken hebben **geen docent** gekoppeld (vakken.docent_id is leeg) en krijgen elk één standaard toetsonderdeel ('Tentamen', weging 100%). Koppel de docenten en verfijn de toetsopbouw via **Vakstructuur** voordat docenten cijfers gaan invoeren; zonder docentkoppeling blijft 'Mijn vakken' leeg.

10. Takenlijst (Studentenzaken)

Tabel taken, model naar Outlook Taken / Microsoft Graph todoTask: titel, omschrijving, startdatum, vervaldatum, status (open | bezig | afgerond), prioriteit (laag | normaal | hoog), afgerond_op, afgerond_door_id. Optionele koppelingen: student_id, toegewezen_aan_id, aangemaakt_door_id, afgerond_door_id — alle vier nullable, zodat een taak blijft bestaan als een student of medewerker verdwijnt.

afgerond_op en afgerond_door_id volgen altijd de status: bij afronden worden beide gezet, bij heropenen beide op null. Dat gebeurt zowel via de afvinkknop als via het bewerkformulier. De afvinker is bewust een eigen veld en niet toegewezen_aan_id: een niet-toegewezen taak mag door iedereen worden opgepakt, en een collega mag andermans taak afronden. Bij taken die vóór deze kolom werden afgerond blijft het veld leeg; dat wordt niet met terugwerkende kracht ingevuld.

'Te laat' is geen kolom. De status wordt afgeleid uit vervaldatum < vandaag én status != afgerond (Taak::isTeLaat()). Sla dit nooit op: anders kan een afgeronde taak in de database als 'te laat' blijven staan.

Toegang uitsluitend voor Studentenzaken en Beheer (rol::magTakenBeheren(), routegroep rol::studentenzaken,beheerder). Er is bewust **geen audit-logging**: een taak is werkverdeling, geen gevoelig persoonsgegeven. Wel blijft zichtbaar wie de taak aanmaakte en aan wie zij is toegewezen.

11. Regulier onderhoud

- **Migraties:** php artisan migrate na een update (reversible; maak eerst een back-up).
- **Cache:** php artisan optimize:clear bij onverwacht gedrag na wijzigingen.
- **Logs:** storage/logs/ (zitten niet in de back-up).
- **Tests:** php artisan test moet groen zijn vóór uitrol.

Deze handleiding wordt bijgewerkt zodra er functies of infrastructuur wijzigen. Controleer de datum onderaan op actualiteit.